학습 교안

YOLO와 ChatGPT를 활용한 생성형 AI 어플리케이션 개발 | 웹 앱 만들기

Streamlit 기초

파이썬 코드로 웹 앱 만들기

예제 앱 12개 · 실제 실행 화면 12장 · 확인 문제로 배우는 학생용 교안 (Streamlit 1.66 에서 모든 예제 실행 확인)

title · write · 위젯 · sidebar · columns · dataframe · chart · session_state · cache · file_uploader · pages · chat · 배포

구성

1~2장 Streamlit 시작하기 · 글자와 숫자 보여 주기

3~5장 입력 위젯 · 화면 배치 · 표와 차트

6~8장 session_state · 캐싱 · 파일 올리고 내려받기

9~10장 여러 페이지 앱 · 채팅 앱과 ChatGPT

11~12장 미니 프로젝트(대시보드 · 문장 유사도) · 인터넷 배포

부록 확인 문제 정답 · 함수 한 장 요약

목 차

이 교안의 사용법 4
학습 순서 4
Streamlit 시작하기 6
1.1 Streamlit 이란? 6
1.2 설치와 실행 6
1.3 첫 번째 앱 7
1.4 실행 원리 : 매번 처음부터 다시 8
글자와 숫자 보여 주기 9
2.1 글자 출력 함수 10
2.2 만능 함수 st.write 11
2.3 숫자 카드 st.metric 과 알림 상자 11
입력 위젯 12
화면 배치하기 15
4.1 배치 함수 정리 16
4.2 with 문으로 넣기 17
4.3 폼(form)은 왜 쓰나? 17
표와 차트 18
5.1 표 보여 주기 19
5.2 차트 19
상태 기억하기 : session_state 21
6.1 session_state 사용법 22
캐싱으로 빠르게 24
파일 올리고 내려받기 27
여러 페이지 앱 30
채팅 앱과 ChatGPT 32
10.1 채팅 화면의 구조 33
10.2 ChatGPT 연결하기 33

미니 프로젝트 35
11.1 타이타닉 생존 대시보드 35
11.2 문장 유사도 분석기 36
인터넷에 배포하기 39
12.1 준비할 파일 39
12.2 배포 순서 39
부록 A. 확인 문제 정답 41
1장 Streamlit 시작하기 41
2장 글자와 숫자 보여 주기 41
3장 입력 위젯 41
4장 화면 배치하기 41
5장 표와 차트 41
6장 상태 기억하기 : session_state 42
7장 캐싱으로 빠르게 42
8장 파일 올리고 내려받기 42
9장 여러 페이지 앱 42
10장 채팅 앱과 ChatGPT 42
11장 미니 프로젝트 43
12장 인터넷에 배포하기 43
부록 B. Streamlit 함수 한 장 요약 44

START

이 교안의 사용법

Streamlit 을 처음 배우는 사람을 위한 실습 중심 교안

이 교안은 함께 나눠 주는 **Streamlit_실습코드** 폴더의 예제를 직접 실행하면서 배우도록 만들었습니다. 각 장의 [실습 코드] 상자는 폴더의 파일과 내용이 **똑같고**, 그림은 그 코드를 실제로 실행한 화면입니다. 코드를 읽기만 하지 말고 꼭 실행해 보고, 값을 바꿔 가며 화면이 어떻게 달라지는지 확인하세요.

표시	의미
[실습 코드] st0n_...py	실습코드 폴더의 예제 파일 — `streamlit run 파일이름` 으로 실행
[터미널]	명령 프롬프트 / VS Code 터미널에 입력하는 명령
그림 n-n	예제를 실제로 실행한 화면
주의 / 기억하기 상자(노랑)	자주 하는 실수와 꼭 기억할 점
더 알아보기 상자(회색)	조금 더 깊은 내용 — 처음에는 건너뛰어도 됨
[실행 결과] · 도전 과제(초록)	실행해서 확인한 값, 스스로 해 볼 과제
확인 문제	각 장 끝 문제 — 정답은 부록 A

학습 순서

장	주제	실습 코드	배우는 것
1	시작하기	st01_hello.py	설치 · 실행 · 재실행 원리
2	글자와 숫자	st02_text.py	title · markdown · write · metric · 알림
3	입력 위젯	st03_widgets.py	button · slider · selectbox · checkbox ...
4	화면 배치	st04_layout.py	sidebar · columns · tabs · expander · form
5	표와 차트	st05_data_chart.py	dataframe · data_editor · bar_chart · pyplot
6	상태 기억	st06_session_state.py	session_state · 콜백 · key
7	캐싱	st07_cache.py	cache_data · cache_resource
8	파일	st08_file_upload.py	file_uploader · stop · download_button
9	여러 페이지	pages_demo/	pages 폴더 · 페이지 간 상태 공유
10	채팅 · ChatGPT	st10_chatbot.py	chat_message · chat_input · secrets
11	미니 프로젝트	st11_dashboard.py, st12_text_analyzer.py	대시보드 · TF-IDF 유사도 앱
12	배포	-	requirements.txt · Community Cloud

실행 환경

Python 3.10 이상과 VS Code 를 준비하고 `pip install streamlit pandas matplotlib scikit-learn` 을 실행합니다. 실습코드 폴더를 VS Code 로 열고 터미널에서 `streamlit run st01_hello.py` 처럼 실행하세요. 이 교안의 모든 예제는 Streamlit 1.66 · pandas 3.0 · Python 3.12 에서 실행해 확인했습니다.

CHAPTER 1

Streamlit 시작하기

Streamlit 이란? · 설치 · 첫 앱 실행 · 실행 원리

학습 목표

- Streamlit 이 무엇이고 왜 쓰는지 말할 수 있다.
- Streamlit 을 설치하고 `streamlit run` 으로 앱을 실행할 수 있다.
- '위젯을 바꾸면 파일 전체가 위에서부터 다시 실행된다'는 실행 원리를 설명할 수 있다.

1.1 Streamlit 이란?

Streamlit 은 파이썬 코드만으로 **웹 앱**을 만드는 라이브러리입니다. HTML·CSS·JavaScript 를 몰라도 `st.title()`, `st.slider()` 처럼 함수를 부르면 화면에 제목과 슬라이더가 나타납니다. 데이터 분석 결과를 대시보드로 보여주거나, 머신러닝 모델·ChatGPT 를 붙인 데모 앱을 만들 때 많이 씁니다.

구분	Jupyter / Colab 노트북	Streamlit 앱
보는 사람	코드를 아는 사람	누구나 (웹 브라우저)
화면	코드 + 출력 결과	버튼·입력창·차트가 있는 웹 페이지
실행	셀 단위로 실행	파일 전체를 위에서 아래로 실행
공유	노트북 파일 전달	웹 주소(URL) 하나로 공유

1.2 설치와 실행

명령 프롬프트(또는 VS Code 터미널)에서 설치합니다. 이 교안은 **Streamlit 1.66** 에서 모든 예제를 실행해 확인했습니다.

[터미널]

```
pip install streamlit
streamlit hello                # 설치 확인용 데모 앱
streamlit run st01_hello.py    # 내 앱 실행
```

주의 : python 으로 실행하면 안 됩니다

Streamlit 앱은 `python st01_hello.py` 가 아니라 **`streamlit run st01_hello.py`** 로 실행합니다. 실행하면 브라우저가 자동으로 열리고 주소는 `http://localhost:8501` 입니다. 끝낼 때는 터미널에서 **Ctrl + C** 를 누릅니다.

> **Colab 에서 해 보고 싶다면?**
>
> Streamlit 은 내 컴퓨터에서 웹 서버를 띄우는 방식이라 Colab 에서는 바로 화면을 볼 수 없습니다. 이 교안은 **내 PC(VS Code) 실행**을 기준으로 합니다. 인터넷에 올리는 방법은 12장(배포)에서 배웁니다.

1.3 첫 번째 앱

[실습 코드] st01_hello.py

```
# st01_hello.py — 1장 : 첫 Streamlit 앱
# 실행: streamlit run st01_hello.py
import streamlit as st

st.title("안녕하세요, Streamlit! 👋")
st.write("파이썬 파일 하나로 웹 앱을 만듭니다.")

name = st.text_input("이름을 입력하세요", "홍길동")
st.write(f"반갑습니다, **{name}** 님!")

if st.button("풍선 날리기"):
    st.balloons()
```

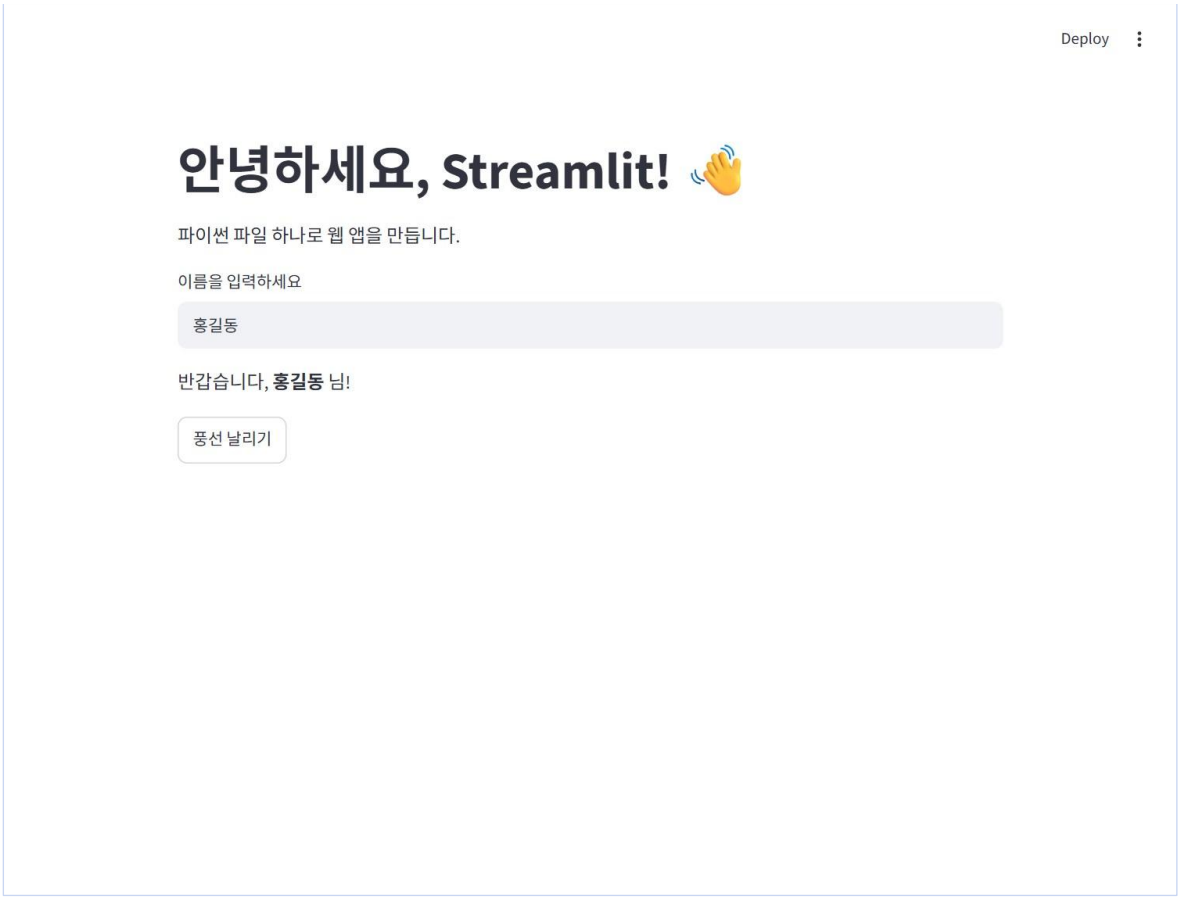


그림 1-1. st01_hello.py 실행 화면

`import streamlit as st` 로 불러오고, `st.함수()` 를 부를 때마다 화면에 요소가 위에서 아래로 쌓입니다. 이름을 바꿔 입력하면 아래 인사말이 바로 바뀌고, 버튼을 누르면 풍선이 날아갑니다.

1.4 실행 원리 : 매번 처음부터 다시

Streamlit 의 가장 중요한 규칙입니다. 사용자가 **입력창에 글자를 넣거나 버튼을 누를 때마다, 파이썬 파일 전체가 맨 위부터 다시 실행**됩니다. 그래서 `name` 변수에는 항상 '지금 입력창에 있는 값'이 들어 있습니다.

[실행 흐름]

```
① 처음 열기      → 파일 전체 실행 → 화면 그리기
② 이름 입력      → 파일 전체 다시 실행 (name = '김철수')
③ 버튼 클릭      → 파일 전체 다시 실행 (st.button(...) 이 이번 한 번만 True)
```

기억하기

버튼 `st.button()` 은 **누른 그 순간의 실행에서만 True** 이고, 다음 실행에서는 다시 False 입니다. 버튼으로 숫자를 계속 늘리고 싶으면 6장의 `session_state` 가 필요합니다.

파일을 저장하면 바로 반영

앱을 켜 둔 채 코드를 고치고 저장하면, 브라우저 오른쪽 위에 'Source file changed' 와 **Rerun** 버튼이 나옵니다. **Always rerun** 을 누르면 저장할 때마다 자동으로 새로고침됩니다.

■ 확인 문제 (정답은 부록 A)

1. Streamlit 앱을 실행하는 명령어는 무엇인가?
2. 입력창의 값을 바꾸면 Streamlit 은 코드의 어느 부분을 다시 실행하는가?

CHAPTER 2

글자와 숫자 보여 주기

title · header · markdown · write · code · latex · metric · 알림 상자

학습 목표

- 제목·본문·코드·수식을 화면에 출력할 수 있다.
- `st.write` 와 `st.metric`, 알림 상자(success/info/warning/error)를 쓸 수 있다.

[실습 코드] st02_text.py

```
# st02_text.py – 2장 : 글자와 숫자 보여 주기
import streamlit as st


st.title("제목 : st.title")
st.header("큰 소제목 : st.header")
st.subheader("작은 소제목 : st.subheader")
st.text("고정폭 텍스트 : st.text")


st.markdown("**굵게**, *기울임*, `코드`, [링크](https://streamlit.io)")
st.markdown("- 목록 1\n- 목록 2")
st.caption("작은 회색 설명 글 : st.caption")


st.code("for i in range(3):\n    print(i)", language="python")
st.latex(r"\hat{y} = ax + b")


st.divider()
st.write("st.write 는 무엇이든 알아서 보여 줍니다 :", 3 + 4, [1, 2, 3], {"a": 1})


col1, col2, col3 = st.columns(3)
col1.metric("기온", "21 °C", "1.2 °C")
col2.metric("습도", "58 %", "-4 %")
col3.metric("방문자", "1,234", "+56")


st.success("성공 메시지")
st.info("안내 메시지")
st.warning("경고 메시지")
st.error("오류 메시지")
```



그림 2-1. st02_text.py 실행 화면

2.1 글자 출력 함수

함수	용도	예
`st.title()`	가장 큰 제목 (페이지에 하나)	앱 이름
`st.header()` / `st.subheader()`	소제목	1장, 1.1절
`st.markdown()`	마크다운 문법 — **굵게**, *기울임*, 목록, 링크	설명 글
`st.text()`	서식 없는 고정폭 글자	로그
`st.caption()`	작은 회색 설명	출처, 도움말
`st.code()`	코드 상자(복사 버튼 포함)	예제 코드
`st.latex()`	수학 수식	회귀식
`st.divider()`	가로 구분선	-

2.2 만능 함수 st.write

`st.write()` 는 넣은 값의 종류를 보고 알아서 보여 줍니다. 문자열은 마크다운으로, 숫자·리스트·딕셔너리는 보기 좋게, pandas DataFrame 은 표로, matplotlib 그림은 그림으로 출력합니다. 처음에는 `st.write` 하나만 기억해도 충분합니다.

> **더 알아보기 : 매직(magic)**
>
> Streamlit 파일에서 변수 이름이나 값만 한 줄에 쓰면(`df` 처럼) `st.write(df)` 를 한 것과 같이 화면에 나옵니다. 노트북에서 마지막 줄이 출력되던 것과 비슷합니다.

2.3 숫자 카드 st.metric 과 알림 상자

`st.metric(라벨, 값, 변화량)` 은 대시보드에서 자주 쓰는 숫자 카드입니다. 변화량이 + 면 초록 ↑, – 면 빨강 ↓ 로 표시됩니다. `st.columns(3)` 으로 칸을 나눠 나란히 놓았습니다(4장).

함수	색	쓰는 때
`st.success()`	초록	작업 완료
`st.info()`	파랑	안내
`st.warning()`	노랑	주의
`st.error()`	빨강	오류

■ 확인 문제 (정답은 부록 A)

1. 표·리스트·숫자·문자열을 모두 출력할 수 있는 함수 하나는?
2. `st.metric('기온', '21 °C', '-1.2 °C')` 의 변화량은 어떤 색으로 표시되는가?

CHAPTER 3

입력 위젯

button · text_input · number_input · slider · selectbox · radio · multiselect · checkbox · date_input

학습 목표

- 사용자에게 값을 입력받는 위젯을 종류별로 쓸 수 있다.
- 위젯의 반환값을 변수에 담아 조건문으로 처리할 수 있다.

위젯(widget)은 사용자가 조작하는 화면 요소입니다. Streamlit 위젯은 **함수가 곧 입력값을 돌려준다**는 점이 편리합니다. `score = st.slider(...)` 라고 쓰면, 슬라이더를 움직일 때마다 앱이 다시 실행되면서 `score` 에 새 값이 들어옵니다.

[실습 코드] st03_widgets.py

```
# st03_widgets.py — 3장 : 입력 위젯
import datetime
import streamlit as st


st.title("입력 위젯 모음")


name = st.text_input("이름", placeholder="이름을 입력하세요")
age = st.number_input("나이", min_value=0, max_value=120, value=20, step=1)
score = st.slider("점수", 0, 100, 70)
grade = st.selectbox("학년", ["1학년", "2학년", "3학년"])
gender = st.radio("성별", ["남", "여"], horizontal=True)
hobbies = st.multiselect("취미", ["독서", "운동", "게임", "음악"], default=["독서"])
agree = st.checkbox("개인정보 수집에 동의합니다")
birthday = st.date_input("생일", datetime.date(2005, 1, 1))
memo = st.text_area("자기소개", height=80)


if st.button("제출", type="primary"):
    if not agree:
        st.warning("동의에 체크해 주세요.")
    else:
        st.success(f"{name}({age}세, {grade}, {gender}) 님 제출 완료!")
        st.write("점수:", score, " / 취미:", ", ".join(hobbies), " / 생일:", birthday)
```

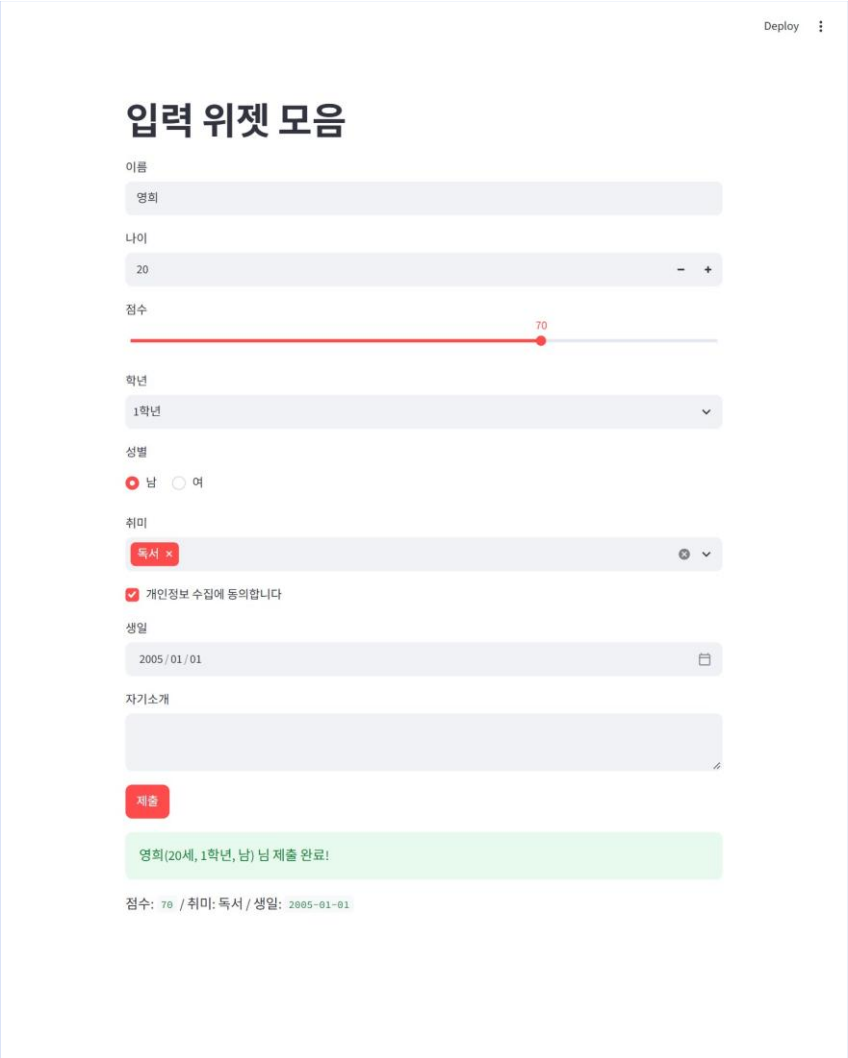


그림 3-1. 동의에 체크하고 [제출] 을 누른 화면

위젯	돌려주는 값	핵심 인자
`st.button('글자')`	누른 순간 True	type='primary' (강조 색)
`st.text_input('라벨')`	문자열	value, placeholder, type='password'
`st.text_area('라벨')`	여러 줄 문자열	height
`st.number_input('라벨')`	숫자	min_value, max_value, value, step
`st.slider('라벨', 최소, 최대, 기본)`	숫자 (기본값이 튜플이면 범위)	(0, 80) → 범위 선택
`st.selectbox('라벨', 목록)`	목록 중 1개	index
`st.radio('라벨', 목록)`	목록 중 1개	horizontal=True
`st.multiselect('라벨', 목록)`	리스트(여러 개)	default
`st.checkbox('라벨')`	True / False	value
`st.date_input('라벨')`	datetime.date	기본 날짜

같은 위젯을 두 번 만들 때

라벨이 같은 위젯을 두 개 만들면 DuplicateWidgetID 오류가 납니다. `st.button('삭제', key='del1')`, `st.button('삭제', key='del2')` 처럼 **key** 를 다르게 주세요.

더 알아보기 : 그 밖의 위젯

`st.toggle`(스위치), `st.color_picker`(색), `st.time_input`(시간), `st.camera_input`(웹캠 사진), `st.feedback`(👍 👎) 등도 있습니다. 사용법은 모두 '변수 = st.위젯(라벨, ...)' 입니다.

■ 확인 문제 (정답은 부록 A)

1. `st.slider('나이 범위', 0, 80, (20, 40))` 의 반환값은 어떤 모양인가?
2. 앞의 st03 예제에서 체크박스를 체크하지 않고 [제출] 을 누르면 어떤 상자가 나오는가?

CHAPTER 4

화면 배치하기

set_page_config · sidebar · columns · tabs · expander · form

학습 목표

- 사이드바·열·탭·접기 영역으로 화면을 나눌 수 있다.
- `st.form` 으로 여러 입력을 모아 한 번에 제출할 수 있다.

[실습 코드] st04_layout.py

```
# st04_layout.py — 4장 : 화면 배치 (sidebar · columns · tabs · expander · form)
import streamlit as st


st.set_page_config(page_title="레이아웃 연습", page_icon="🧩", layout="wide")
st.title("레이아웃 연습")


# 1) 사이드바
with st.sidebar:
    st.header("설정")
    color = st.selectbox("테마 색", ["파랑", "초록", "빨강"])
    size = st.slider("글자 크기", 10, 40, 20)


# 2) 열 나누기 (비율 2:1)
left, right = st.columns([2, 1])
left.subheader("왼쪽 넓은 칸")
left.write(f"선택한 색: {color}, 크기: {size}")
right.subheader("오른쪽 좁은 칸")
right.button("오른쪽 버튼")


# 3) 탭
tab1, tab2 = st.tabs(["📈 차트", "📋 표"])
with tab1:
    st.line_chart([1, 3, 2, 5, 4])
with tab2:
    st.table({"이름": ["가", "나"], "점수": [90, 85]})


# 4) 접었다 펴기
with st.expander("자세히 보기"):
    st.write("평소에는 숨겨 두고 필요할 때 펼쳐 보는 영역입니다.")


# 5) 폼 : 제출 버튼을 누를 때만 한 번에 실행
with st.form("login"):
    uid = st.text_input("아이디")
    pw = st.text_input("비밀번호", type="password")
    ok = st.form_submit_button("로그인")
if ok:
    st.write(f"{uid} 님 로그인 시도 (비밀번호 {len(pw)}자리)")
```

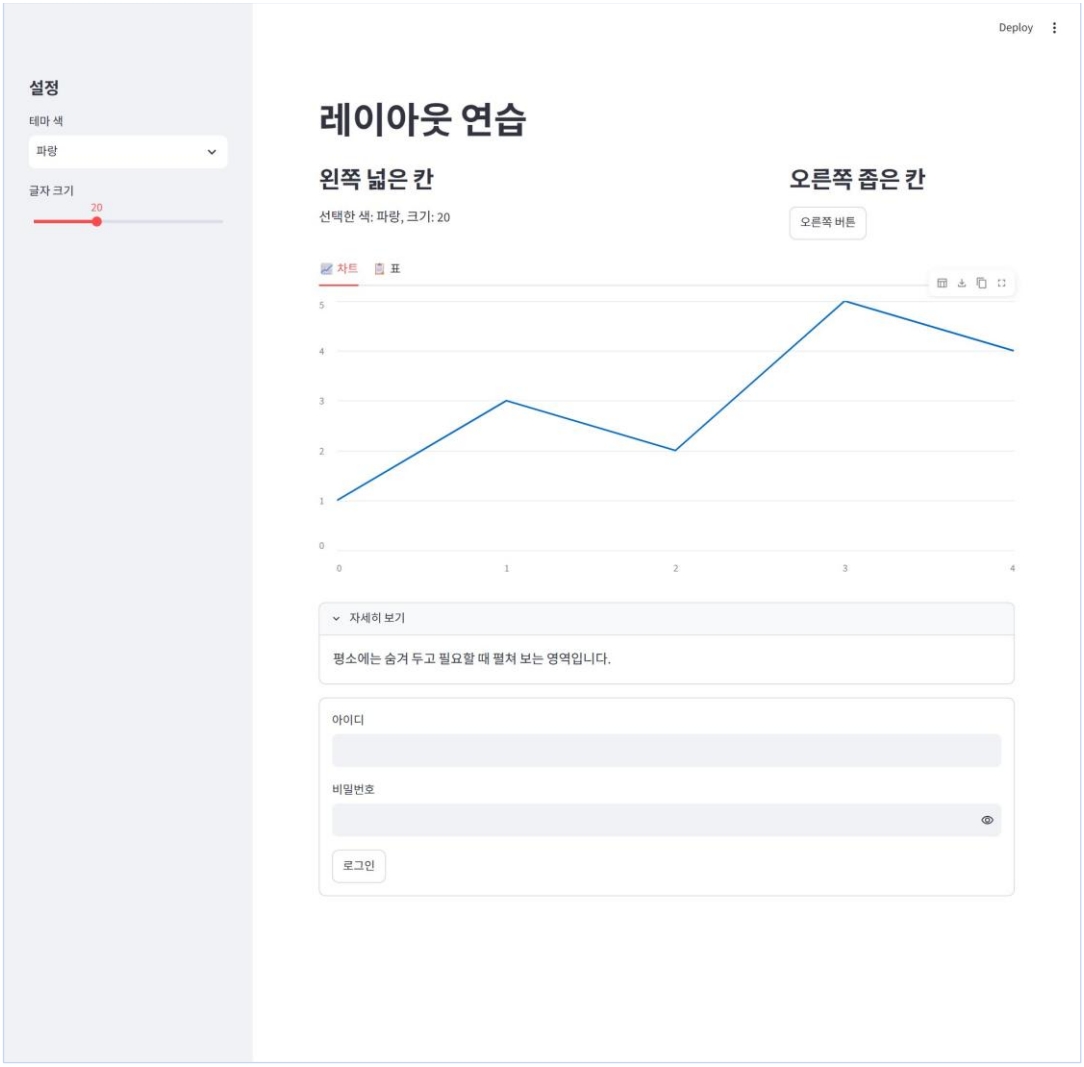


그림 4-1. 사이드바 · 2:1 열 · 탭 · 펼친 expander · 폼 (layout='wide')

4.1 배치 함수 정리

함수	모양	쓰는 법
`st.set_page_config()`	탭 제목·아이콘·넓은 화면	파일 맨 처음에 한 번
`st.sidebar`	왼쪽 접이식 메뉴	`with st.sidebar:` 또는 `st.sidebar.slider()`
`st.columns(n)` / `st.columns([2, 1])`	가로로 n칸 / 비율	`a, b = st.columns(2)` → `a.write()`
`st.tabs([...])`	탭 메뉴	`t1, t2 = st.tabs(['A', 'B'])`
`st.expander('제목')`	접었다 펴는 상자	`with st.expander('...'):`
`st.container()`	보이지 않는 묶음 상자	border=True 로 테두리
`st.form('이름')`	입력 묶음 + 제출 버튼	`st.form_submit_button()` 필수

4.2 with 문으로 넣기

`with 영역:` 아래에 들여쓴 코드는 모두 그 영역 안에 그려집니다. `with st.sidebar:` 안의 위젯은 사이드바에, `with tab1:` 안의 차트는 첫 번째 탭에 나타납니다. 열 객체에 바로 `left.write()` 처럼 써도 됩니다.

4.3 폼(form)은 왜 쓰나?

보통 위젯은 값을 바꿀 때마다 앱이 다시 실행됩니다. 로그인처럼 아이디·비밀번호를 **다 입력한 뒤 한 번만** 처리하고 싶을 때 `st.form` 으로 묶습니다. 폼 안에서는 [로그인] 버튼을 누를 때까지 다시 실행되지 않습니다.

> **주의**
>
> `st.set_page_config()` 는 다른 st 함수보다 **먼저** 한 번만 불러야 합니다. 폼 안에는 `st.button` 대신 `st.form_submit_button` 을 씁니다.

■ 확인 문제 (정답은 부록 A)

1. 화면을 3:1 비율의 두 칸으로 나누는 코드는?
2. 로그인 입력처럼 '다 입력한 뒤 한 번에' 처리하려면 무엇으로 묶는가?

CHAPTER 5

표와 차트

dataframe · data_editor · metric · bar/line_chart · pyplot

학습 목표

- pandas DataFrame 을 표로 보여 주고 직접 편집하게 할 수 있다.
- Streamlit 기본 차트와 matplotlib 그림을 앱에 넣을 수 있다.

데이터 분석 수업에서 쓴 타이타닉 데이터(891명)를 `data/titanic.csv` 로 사용합니다.

[실습 코드] st05_data_chart.py

```
# st05_data_chart.py — 5장 : 표와 차트
from pathlib import Path
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import streamlit as st


DATA = Path(__file__).parent / "data" / "titanic.csv"
plt.rc("font", family="Malgun Gothic"); plt.rcParams["axes.unicode_minus"] = False


st.title("타이타닉 데이터 살펴보기")
df = pd.read_csv(DATA)


st.subheader("1. 표 : st.dataframe")
st.dataframe(df.head(10), width="stretch")


st.subheader("2. 요약 숫자 : st.metric")
c1, c2, c3 = st.columns(3)
c1.metric("승객 수", f"{len(df):,}명")
c2.metric("생존율", f"{df['survived'].mean():.1%}")
c3.metric("평균 나이", f"{df['age'].mean():.1f}세")


st.subheader("3. Streamlit 기본 차트")
by_class = df.groupby("class")["survived"].mean().rename("생존율")
st.bar_chart(by_class)


rng = np.random.default_rng(0)
st.line_chart(pd.DataFrame(rng.normal(size=(30, 2)).cumsum(0), columns=["A", "B"]))


st.subheader("4. matplotlib 그림 : st.pyplot")
fig, ax = plt.subplots(figsize=(6, 3))
df["age"].dropna().plot.hist(bins=20, ax=ax, color="#1f77b4")
ax.set_title("나이 분포"); ax.set_xlabel("나이")
st.pyplot(fig)


st.subheader("5. 표 직접 고치기 : st.data_editor")
edited = st.data_editor(pd.DataFrame({"과목": ["국어", "수학"], "점수": [80, 90]}), num_rows="dynamic")
st.write("평균 점수:", edited["점수"].mean())
```



그림 5-1. st05_data_chart.py 실행 화면 (위쪽 일부)

5.1 표 보여 주기

함수	특징
`st.dataframe(df)`	스크롤·정렬·검색 가능한 표. 열 머리글을 누르면 정렬
`st.table(df)`	고정된 표(작은 표에 적합)
`st.data_editor(df)`	엑셀처럼 칸을 고칠 수 있는 표 — 고친 결과를 DataFrame 으로 돌려줌
`width='stretch'`	표를 화면 폭에 맞춤 (예전 `use_container_width=True` 는 곧 사라짐)

5.2 차트

`st.bar_chart`, `st.line_chart`, `st.area_chart`, `st.scatter_chart` 는 DataFrame 이나 Series 를 넣으면 바로 그려 주고, 마우스를 올리면 값이 보이는 **인터랙티브 차트**입니다. 인덱스가 x축, 열이 y축이 됩니다. 예제에서 `groupby('class')['survived'].mean()` 결과(First 0.63, Second 0.47, Third 0.24)를 그대로 넣었습니다.

matplotlib·seaborn 으로 그린 그림은 `st.pyplot(fig)` 로 넣습니다. 한글 제목이 깨지면 `plt.rc('font', family='Malgun Gothic')` 을 먼저 실행합니다(Windows 기준).

> **더 알아보기 : 다른 차트 라이브러리**
>
> `st.plotly_chart`(Plotly), `st.altair_chart`(Altair), `st.map`(위도·경도 점 지도)도 있습니다. Plotly 는 `pip install plotly` 후 사용합니다.

■ 확인 문제 (정답은 부록 A)

1. 사용자가 표의 값을 직접 고치게 하려면 어떤 함수를 쓰는가?
2. matplotlib 으로 만든 fig 를 앱에 넣는 함수는?

CHAPTER 6

상태 기억하기 : session_state

재실행되어도 값 유지 · 카운터 · 콜백(on_click) · 위젯 key

학습 목표

- 일반 변수가 버튼을 누를 때마다 초기화되는 이유를 설명할 수 있다.
- `st.session_state` 로 값을 기억하는 카운터와 할 일 목록을 만들 수 있다.

1장에서 배운 것처럼 앱은 매번 처음부터 다시 실행됩니다. 그래서 `count = 0` 은 버튼을 누를 때마다 다시 0이 됩니다. **브라우저 탭 하나마다 따로 유지되는 저장소**인 `st.session_state` 에 값을 넣어 두면 재실행되어도 남아 있습니다.

[실습 코드] st06_session_state.py

```
# st06_session_state.py — 6장 : 상태 기억하기 (session_state)
import streamlit as st


st.title("session_state 로 값 기억하기")


# ❌ 잘못된 카운터 : 버튼을 누를 때마다 파일 전체가 다시 실행되어 0으로 돌아감
count_wrong = 0
if st.button("잘못된 +1"):
    count_wrong += 1
st.write("잘못된 카운터:", count_wrong)


st.divider()


# ☑ 올바른 카운터 : session_state 는 재실행되어도 값이 남아 있음
if "count" not in st.session_state:
    st.session_state.count = 0


col1, col2 = st.columns(2)
if col1.button("+1"):
    st.session_state.count += 1
if col2.button("초기화"):
    st.session_state.count = 0
st.metric("올바른 카운터", st.session_state.count)


st.divider()


# 콜백(on_click) + 위젯 key 로 할 일 목록 만들기
if "todos" not in st.session_state:
    st.session_state.todos = []


def add_todo():
    text = st.session_state.new_todo.strip()
    if text:
        st.session_state.todos.append(text)
    st.session_state.new_todo = ""          # 입력창 비우기


st.text_input("할 일", key="new_todo")
st.button("추가", on_click=add_todo)
```

```
for i, t in enumerate(st.session_state.todos, 1):
    st.write(f"{i}. {t}")
```

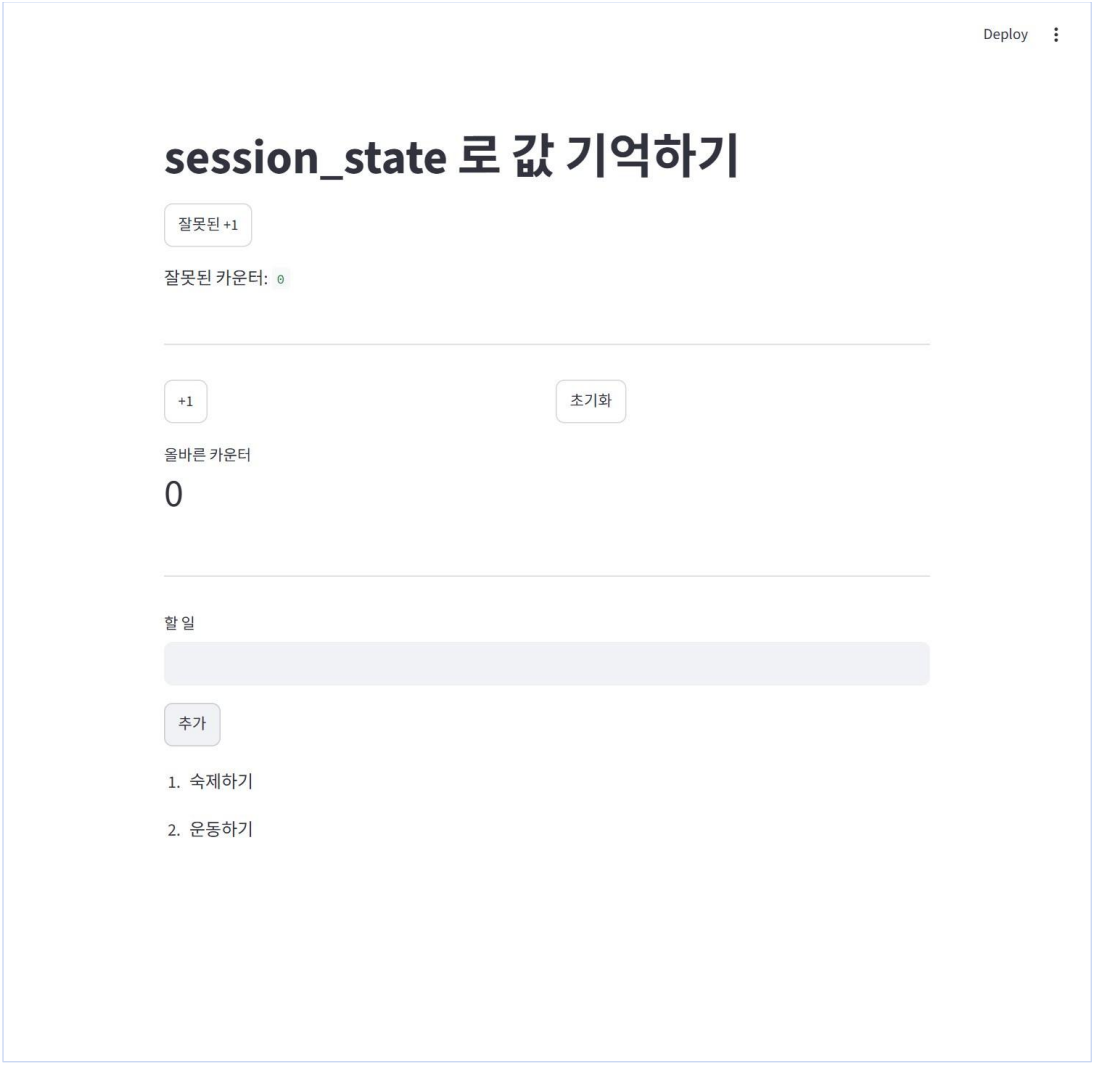


그림 6-1. [+1] 3번, [잘못된 +1] 1번, 할 일 2개를 추가한 화면

[실행 결과] 확인

- [잘못된 +1] 은 몇 번을 눌러도 **최대 1** 입니다. 누른 그 실행에서만 0 → 1 이 되고, 다음 실행에서 다시 0 부터 시작합니다.
- [+1] 을 3번 누르면 올바른 카운터는 **3** 입니다. (자동 테스트로 확인)

6.1 session_state 사용법

코드	의미
`if 'count' not in st.session_state:`	처음 실행일 때만
`st.session_state.count = 0`	초기값 저장 (딕셔너리처럼 `st.session_state['count']` 도 가능)
`st.session_state.count += 1`	값 바꾸기 — 다음 실행에도 유지
`st.text_input('할 일', key='new_todo')`	위젯 값이 `st.session_state.new_todo` 에 자동 저장

코드	의미
`st.button('추가', on_click=add_todo)`	버튼을 누르면 **화면을 다시 그리기 전에** add_todo 실행 (콜백)

> **콜백을 쓰는 이유**
>
> 할 일 예제에서 입력창을 비우려면 `st.session_state.new_todo = ''` 를 해야 합니다. 그런데 입력창이 이미 그려진 뒤에 이 값을 바꾸면 오류가 납니다. **on_click 콜백** 은 다음 실행이 시작되기 전에 실행되므로 안전하게 바꿀 수 있습니다.

■ 확인 문제 (정답은 부록 A)

1. 일반 변수 `count = 0` 으로 만든 카운터가 1 이상 올라가지 않는 이유는?
2. 위젯에 key='nick' 을 주면 그 값은 어디에서 꺼낼 수 있는가?

CHAPTER 7

캐싱으로 빠르게

st.cache_data · st.cache_resource · 캐시 비우기

학습 목표

- 매번 재실행되는 구조에서 캐싱이 왜 필요한지 설명할 수 있다.
- `@st.cache_data` 로 데이터 불러오기를 한 번만 하도록 만들 수 있다.

앱이 매번 처음부터 실행되므로, 큰 CSV 를 읽는 코드도 버튼을 누를 때마다 다시 실행됩니다. 함수 위에 `@st.cache_data` 를 붙이면 **같은 인자로 부른 결과를 저장해 두었다가** 다음부터는 바로 돌려줍니다.

[실습 코드] st07_cache.py

```
# st07_cache.py – 7장 : 캐싱으로 빠르게
import time
from pathlib import Path
import pandas as pd
import streamlit as st


DATA = Path(__file__).parent / "data" / "titanic.csv"
st.title("st.cache_data 효과 보기")


@st.cache_data                      # 같은 인자로 다시 부르면 저장된 결과를 바로 돌려줌
def load_data(path):
    time.sleep(2)                   # 오래 걸리는 작업(큰 파일, DB 조회)을 흉내
    return pd.read_csv(path)


start = time.time()
df = load_data(DATA)
st.write(f"불러오기 시간: **{time.time() - start:.2f}초** (처음 2초, 다음부터 거의 0초)")
st.dataframe(df.head())


if st.button("다시 실행해 보기"):
    st.rerun()
if st.button("캐시 비우기"):
    load_data.clear()
    st.rerun()
```



	survived	pclass	sex	age	sibsp	parch	fare	embarked	class	who	adult_male
0	0	3	male	22	1	0	7.25	S	Third	man	☑
1	1	1	female	38	1	0	71.2833	C	First	woman	☐
2	1	3	female	26	0	0	7.925	S	Third	woman	☐
3	1	1	female	35	1	0	53.1	S	First	woman	☐
4	0	3	male	35	0	0	8.05	S	Third	man	☑

그림 7-1. [다시 실행해 보기] 를 누른 뒤 — 불러오기 0.00초

> **[실행 결과] 확인**
>
> 자동 테스트에서 처음 실행은 **2.41초**, 두 번째 실행은 **0.01초**였습니다. [캐시 비우기] 를 누르면 다시 2초가 걸립니다.

데코레이터	저장하는 것	예
`@st.cache_data`	함수가 돌려준 **데이터**(복사본)	CSV·엑셀 읽기, API 응답, 계산 결과
`@st.cache_resource`	한 번 만든 **객체** 자체(공유)	머신러닝 모델, DB 연결, OpenAI 클라이언트

> **주의**
>
> `@st.cache_data(ttl=600)` 처럼 ttl(초)을 주면 10분이 지나면 새로 불러옵니다. 실시간으로 바뀌는 데이터(주가 등)에 씁니다.

■ 확인 문제 (정답은 부록 A)

1. 학습된 Keras 모델을 한 번만 불러오려면 어떤 데코레이터를 쓰는가?
2. `@st.cache_data` 함수의 저장된 결과를 지우는 방법은?

CHAPTER 8

파일 올리고 내려받기

file_uploader · st.stop · download_button

학습 목표

- 사용자가 올린 CSV 를 pandas 로 읽어 분석할 수 있다.
- 분석 결과를 파일로 내려받게 할 수 있다.

[실습 코드] st08_file_upload.py

```
# st08_file_upload.py — 8장 : 파일 올리고 내려받기
import io
import pandas as pd
import streamlit as st


st.title("CSV 분석기")
st.caption("CSV 파일을 올리면 요약 통계와 그래프를 보여 줍니다. (실습: data/titanic.csv)")


file = st.file_uploader("CSV 파일 선택", type=["csv"])
if file is None:
    st.info("왼쪽 위 'Browse files' 로 파일을 올려 주세요.")
    st.stop()                                   # 아래 코드는 실행하지 않음


df = pd.read_csv(file)
st.success(f"{file.name} : {df.shape[0]}행 × {df.shape[1]}열")
st.dataframe(df.head())


num_cols = df.select_dtypes("number").columns.tolist()
col = st.selectbox("분석할 숫자 열", num_cols)
st.write(df[col].describe())
st.bar_chart(df[col].value_counts().sort_index())


# 결과 내려받기
summary = df.describe().T
buf = io.StringIO(); summary.to_csv(buf)
st.download_button("요약 통계 CSV 내려받기", buf.getvalue().encode("utf-8-sig"),
                   file_name="summary.csv", mime="text/csv")
```

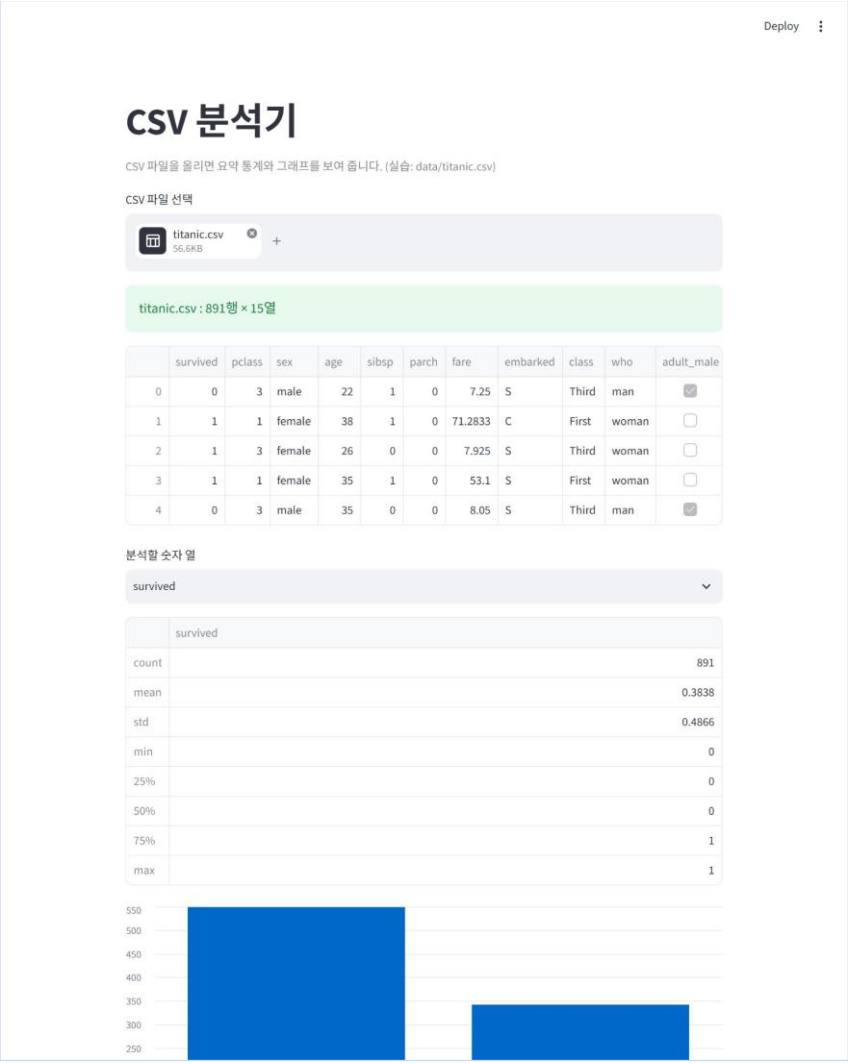


그림 8-1. titanic.csv 를 올린 화면 (891행 × 15열)

코드	설명
`file = st.file_uploader('CSV', type=['csv'])`	올리기 전에는 None, 올리면 파일 객체
`if file is None: st.stop()`	아래 코드를 실행하지 않고 멈춤 — 파일 없이 오류 나는 것을 막음
`pd.read_csv(file)`	파일 객체를 그대로 pandas 에 넣으면 됨
`df.select_dtypes('number')`	숫자 열만 골라 selectbox 목록으로
`st.download_button(라벨, 데이터, file_name=...)`	버튼을 누르면 파일 저장

엑셀에서 한글이 깨지면

CSV 를 엑셀로 열 때 한글이 깨지면 `encoding='utf-8-sig'` 로 저장합니다. 예제의 `.encode('utf-8-sig')` 가 그 역할입니다.

> **더 알아보기**
>
> `st.file_uploader(..., accept_multiple_files=True)` 로 여러 파일을 한 번에, `type=['png', 'jpg']` 로 이미지를 받아 `st.image(file)` 로 보여 줄 수 있습니다. YOLO 수업의 이미지 탐지 결과를 보여 주는 앱도 같은 방식입니다.

■ 확인 문제 (정답은 부록 A)

1. 파일을 아직 올리지 않았을 때 아래 분석 코드가 실행되지 않게 하려면?
2. 다운로드 버튼에 넘기는 데이터는 어떤 형태여야 하는가?

CHAPTER 9

여러 페이지 앱

pages/ 폴더 · 페이지 간 session_state 공유

학습 목표

- pages 폴더 방식으로 여러 페이지 앱을 만들 수 있다.
- 페이지를 옮겨도 session_state 가 유지됨을 확인한다.

메인 파일과 같은 위치에 pages 폴더를 만들고 .py 파일을 넣으면, Streamlit 이 자동으로 왼쪽 사이드바에 페이지 메뉴를 만들어 줍니다. 파일 이름 앞의 숫자로 순서를, 밑줄 뒤 글자로 메뉴 이름을 정합니다.

[폴더 구조]

```
pages_demo/
├── Home.py              ← streamlit run pages_demo/Home.py
└── pages/
    ├── 1_계산기.py      ← 메뉴 이름 '계산기'
    └── 2_소개.py        ← 메뉴 이름 '소개'
```

[실습 코드] pages_demo/Home.py

```
# pages_demo/Home.py — 9장 : 여러 페이지 앱 (실행: streamlit run pages_demo/Home.py)
# 같은 폴더의 pages/ 안에 있는 .py 파일이 자동으로 왼쪽 메뉴에 페이지로 추가됩니다.
import streamlit as st


st.set_page_config(page_title="멀티페이지 데모", page_icon="🏠")
st.title("🏠 홈")
st.write("왼쪽 사이드바에서 페이지를 고르세요.")

if "nickname" not in st.session_state:
    st.session_state.nickname = "손님"
st.session_state.nickname = st.text_input("닉네임 (다른 페이지에서도 유지됨)", st.session_state.nickname)
```

[실습 코드] pages_demo/pages/1_계산기.py

```
import streamlit as st


st.title("🧮 계산기")
st.write(f"{st.session_state.get('nickname', '손님')} 님, 숫자 두 개를 입력하세요.")
a = st.number_input("A", value=3)
b = st.number_input("B", value=4)
op = st.radio("연산", ["+", "-", "×", "÷"], horizontal=True)
res = {"+": a + b, "-": a - b, "×": a * b, "÷": a / b if b else float("nan")}[op]
st.metric("결과", res)
```

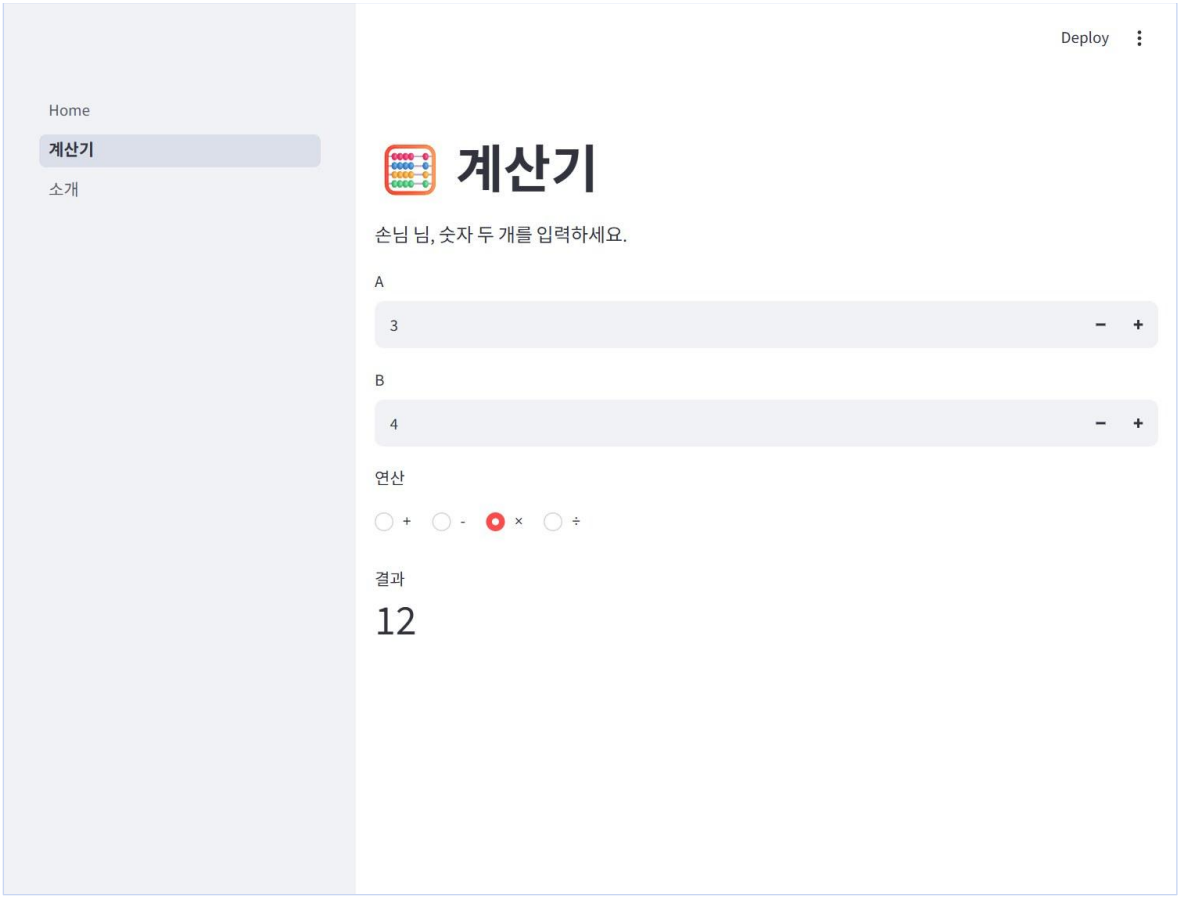


그림 9-1. 사이드바 메뉴에서 '계산기' 를 고르고 × 를 선택 (3 × 4 = 12)

> **더 알아보기 : st.navigation**
>
> 페이지 목록을 코드로 정하고 싶으면 `pg = st.navigation([st.Page('home.py', title='홈'), st.Page('calc.py', title='계산기')])` 후 `pg.run()` 을 씁니다. 로그인 여부에 따라 메뉴를 바꾸는 등 더 자유롭게 구성할 수 있습니다.

■ 확인 문제 (정답은 부록 A)

1. pages 폴더의 파일 '3_결과보기.py' 는 메뉴에 어떤 이름으로, 몇 번째로 나오는가?

CHAPTER 10

채팅 앱과 ChatGPT

chat_message · chat_input · secrets · OpenAI API

학습 목표

- `st.chat_message` 와 `st.chat_input` 으로 채팅 화면을 만든다.
- 대화 기록을 session_state 에 저장하고, API 키를 secrets 로 안전하게 다룬다.

이번 과정의 목표인 'ChatGPT 를 활용한 앱'의 뼈대입니다. 먼저 키 없이 동작하는 **따라 말하기 봇**을 만들고, 키를 넣으면 같은 화면이 ChatGPT 와 대화하도록 바뀝니다.

[실습 코드] st10_chatbot.py

```
# st10_chatbot.py — 10장 : 채팅 화면과 ChatGPT 연결
# OpenAI 키가 없으면 '따라 말하기' 봇으로 동작합니다.
# 키 설정 : .streamlit/secrets.toml 에  OPENAI_API_KEY = "sk-..."   (깃허브에 올리지 말 것!)
import streamlit as st


st.title("💬 나의 첫 챗봇")


try:
    api_key = st.secrets["OPENAI_API_KEY"]
except Exception:
    api_key = None
mode = "ChatGPT" if api_key else "따라 말하기(키 없음)"
st.caption(f"현재 모드: {mode}")


if "messages" not in st.session_state:
    st.session_state.messages = [{"role": "assistant", "content": "안녕하세요! 무엇이든 물어보세요."}]


for m in st.session_state.messages:                # 지난 대화 다시 그리기
    with st.chat_message(m["role"]):
        st.write(m["content"])


def ask_gpt(messages):
    from openai import OpenAI                        # pip install openai
    client = OpenAI(api_key=api_key)
    res = client.chat.completions.create(model="gpt-4o-mini", messages=messages)
    return res.choices[0].message.content


if prompt := st.chat_input("메시지를 입력하세요"):
    st.session_state.messages.append({"role": "user", "content": prompt})
    with st.chat_message("user"):
        st.write(prompt)
    with st.chat_message("assistant"):
        answer = ask_gpt(st.session_state.messages) if api_key else f"'{prompt}' 라고 하셨군요! 👍"
        st.write(answer)
    st.session_state.messages.append({"role": "assistant", "content": answer})
```

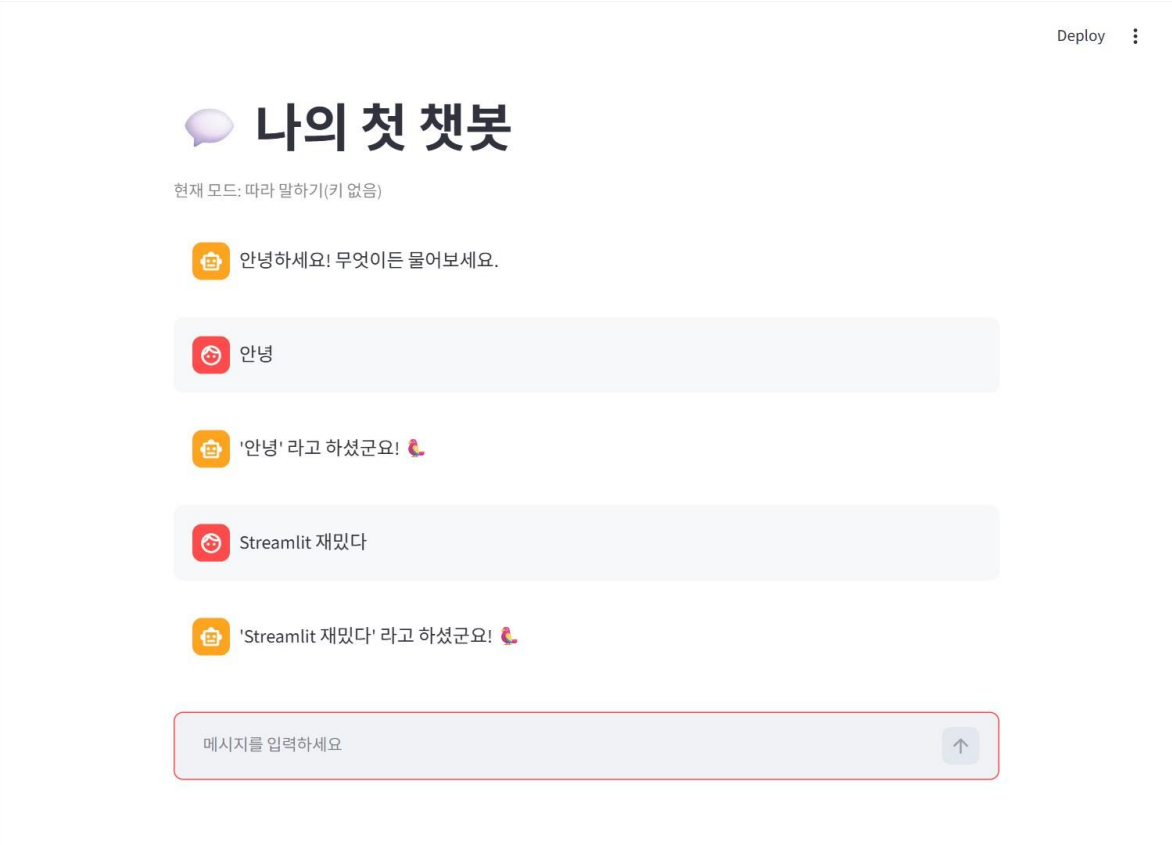


그림 10-1. 키 없이 실행한 따라 말하기 모드

10.1 채팅 화면의 구조

코드	역할
`st.session_state.messages`	대화 기록 리스트 — [{'role': 'user', 'content': '...'}, ...]
`for m in messages: with st.chat_message(m['role']):`	재실행될 때마다 지난 대화를 다시 그림
`if prompt := st.chat_input('...'):`	화면 아래 입력창. 입력하면 문자열, 아니면 None
`ask_gpt(messages)`	대화 기록 전체를 OpenAI 에 보내 답을 받음

:= (바다코끼리 연산자)

`if prompt := st.chat_input():` 은 '입력값을 prompt 에 담고, 그 값이 있으면' 이라는 뜻으로 `prompt = st.chat_input()` + `if prompt:` 두 줄을 한 줄로 쓴 것입니다(Python 3.8+).

10.2 ChatGPT 연결하기

[터미널]

```
pip install openai
```

[.streamlit/secrets.toml] ← 앱 파일과 같은 폴더의 .streamlit 폴더 안

```
OPENAI_API_KEY = "sk-여기에_내_키"
```

> **API 키는 절대 코드에 쓰지 마세요**
>
> - 키를 코드에 직접 쓰고 GitHub 에 올리면 다른 사람이 내 요금으로 사용할 수 있습니다.
> - `secrets.toml` 은 `.gitignore` 에 추가해 업로드되지 않게 합니다.
> - API 사용은 유료입니다. 수업용 키의 사용량 한도를 확인하세요.

키를 저장하고 앱을 다시 실행하면 캡션이 '현재 모드: ChatGPT' 로 바뀌고, 지금까지의 대화 기록(messages) 전체를 보내므로 앞 대화를 기억하는 챗봇이 됩니다. (이 교안의 확인 환경에는 키가 없어 따라 말하기 모드만 실행해 확인했습니다.)

■ 확인 문제 (정답은 부록 A)

1. 챗봇이 앞의 대화를 기억하게 하려면 OpenAI 에 무엇을 보내야 하는가?
2. API 키를 코드에 직접 쓰지 않고 넣는 Streamlit 의 방법은?

CHAPTER 11

미니 프로젝트

① 타이타닉 대시보드 ② 문장 유사도 분석기

학습 목표

- 지금까지 배운 위젯·배치·차트·캐싱을 합쳐 대시보드를 만든다.
- NLP 수업의 TF-IDF·코사인 유사도를 Streamlit 앱으로 만든다.

11.1 타이타닉 생존 대시보드

사이드바에서 성별·객실 등급·나이를 고르면 숫자 카드와 차트가 바로 바뀌는 대시보드입니다. `@st.cache_data`(7장), 사이드바·열(4장), metric·bar_chart(5장)를 모두 씁니다.

[실습 코드] st11_dashboard.py

```
# st11_dashboard.py — 11장 미니 프로젝트 ① : 타이타닉 대시보드
from pathlib import Path
import pandas as pd
import streamlit as st


st.set_page_config(page_title="타이타닉 대시보드", page_icon="🚢", layout="wide")
DATA = Path(__file__).parent / "data" / "titanic.csv"


@st.cache_data
def load():
    return pd.read_csv(DATA)


df = load()


with st.sidebar:
    st.header("🔎 필터")
    sex = st.multiselect("성별", ["male", "female"], default=["male", "female"])
    pclass = st.multiselect("객실 등급", [1, 2, 3], default=[1, 2, 3])
    age = st.slider("나이 범위", 0, 80, (0, 80))


# 나이를 모르는 승객(177명)은 나이 조건과 상관없이 포함
f = df[df["sex"].isin(sex) & df["pclass"].isin(pclass) & (df["age"].between(*age) | df["age"].isna())]


st.title("🚢 타이타닉 생존 대시보드")
c1, c2, c3, c4 = st.columns(4)
c1.metric("선택된 승객", f"{len(f):,}명")
c2.metric("생존자", f"{int(f['survived'].sum()):,}명")
c3.metric("생존율", f"{f['survived'].mean():.1%}" if len(f) else "-")
c4.metric("평균 요금", f"${f['fare'].mean():.1f}" if len(f) else "-")


left, right = st.columns(2)
with left:
    st.subheader("성별 생존율")
    st.bar_chart(f.groupby("sex")["survived"].mean())
with right:
    st.subheader("객실 등급별 생존율")
```

```
    st.bar_chart(f.groupby("pclass")["survived"].mean())

with st.expander("선택된 데이터 보기"):
    st.dataframe(f, width="stretch")
```

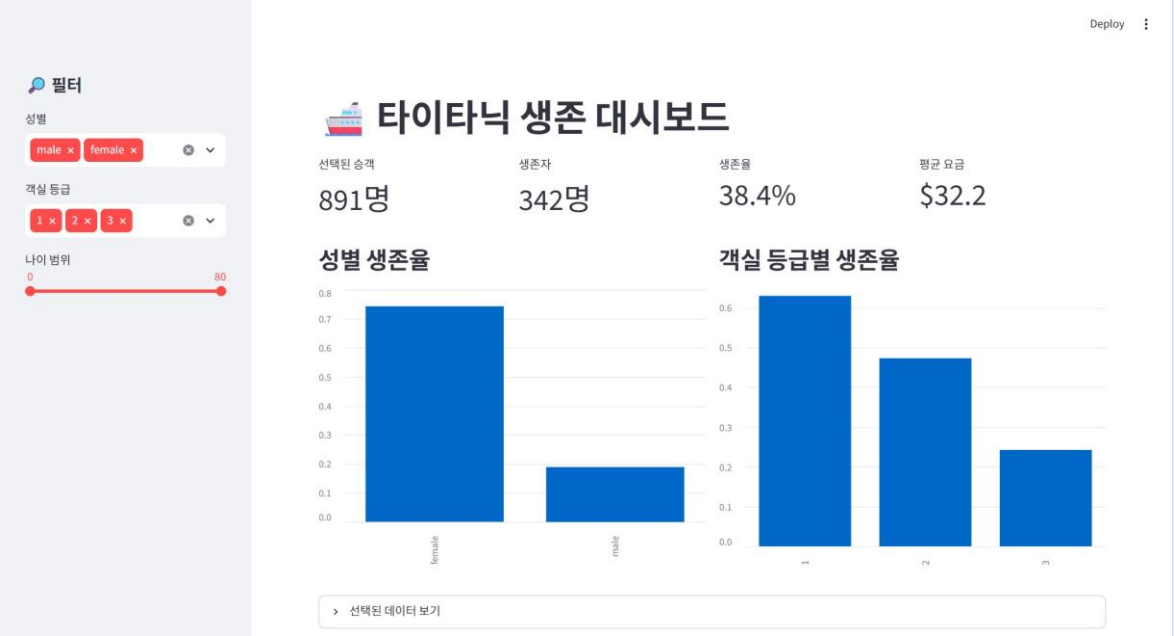


그림 11-1. 타이타닉 대시보드 (전체 선택)

필터	선택된 승객	생존율	평균 요금
전체	891명	38.4%	$32.2
여성만	314명	74.2%	$44.5
여성 + 1등석	94명	96.8%	$106.1

위 표는 자동 테스트로 필터를 바꿔 가며 확인한 값입니다. 나이를 모르는 승객 177명은 나이 범위와 상관없이 포함되도록 `| df['age'].isna()` 조건을 넣었습니다 — 이 줄을 빼면 기본 화면에서 714명만 보입니다.

11.2 문장 유사도 분석기

NLP 수업에서 배운 TF-IDF 와 코사인 유사도를 앱으로 만듭니다. 문장 목록과 질문 문장을 입력하면 가장 비슷한 문장을 찾아 줍니다.

[실습 코드] st12_text_analyzer.py

```
# st12_text_analyzer.py — 11장 미니 프로젝트 ② : 문장 유사도 분석기 (NLP 수업 연결)
import pandas as pd
import streamlit as st
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity


st.title("📝 문장 유사도 분석기")
st.caption("TF-IDF 로 문장을 벡터로 바꾸고, 코사인 유사도로 가장 비슷한 문장을 찾습니다.")


default = "I like dog\nYou like dog\nI hate bug\nA dog in a dog house is big"
```

```
docs = [s for s in st.text_area("문장 목록 (한 줄에 하나)", default, height=130).splitlines() if s.strip()]
query = st.text_input("비교할 문장", "I love my dog")


if docs and query:
    tv = TfidfVectorizer(token_pattern=r"(?u)\b\w+\b")
    X = tv.fit_transform(docs + [query])
    sims = cosine_similarity(X[-1], X[:-1]).ravel()
    result = pd.DataFrame({"문장": docs, "유사도": sims.round(3)}).sort_values("유사도", ascending=False)


    best = result.iloc[0]
    st.success(f"가장 비슷한 문장: **{best['문장']}** (유사도 {best['유사도']})")
    st.bar_chart(result.set_index("문장")["유사도"])
    with st.expander("TF-IDF 행렬 보기"):
        st.dataframe(pd.DataFrame(X.toarray(), columns=tv.get_feature_names_out(),
                                  index=docs + ["(질문)"]).round(2))
```



그림 11-2. 'I love my dog' 과 가장 비슷한 문장은 'I like dog' (유사도 0.387)

도전 과제

- 대시보드에 '탑승 항구(embark_town)' 필터를 추가해 보세요.
- 유사도 분석기에 `st.file_uploader` 를 붙여 txt 파일의 문장들과 비교해 보세요.
- 10장 챗봇 화면에 사이드바 `st.slider('temperature', 0.0, 2.0, 1.0)` 를 추가해 답변 스타일을 바꿔 보세요.

■ 확인 문제 (정답은 부록 A)

1. 대시보드에서 필터를 바꿀 때마다 CSV 를 다시 읽지 않게 해 주는 코드는?

CHAPTER 12

인터넷에 배포하기

requirements.txt · GitHub · Streamlit Community Cloud · secrets

학습 목표

- 내 앱을 무료로 인터넷에 올리는 절차를 안다.
- 배포할 때 필요한 파일과 비밀 키 관리 방법을 안다.

Streamlit Community Cloud(share.streamlit.io)는 GitHub 저장소에 있는 Streamlit 앱을 무료로 웹에 올려 주는 서비스입니다. 올리면 `https://내앱이름.streamlit.app` 같은 주소가 생겨 누구에게나 링크로 보여 줄 수 있습니다.

12.1 준비할 파일

[저장소 구조 예]

```
my-app/
├── app.py                ← 메인 파일
├── requirements.txt      ← 필요한 라이브러리 목록
├── data/titanic.csv
└── .gitignore            ← .streamlit/secrets.toml 을 꼭 넣기
```

[requirements.txt]

```
streamlit
pandas
matplotlib
scikit-learn
openai
```

12.2 배포 순서

1. GitHub 에 새 저장소를 만들고 위 파일들을 올린다(secrets.toml 은 올리지 않는다).
2. share.streamlit.io 에 GitHub 계정으로 로그인한다.
3. [Create app] → 저장소 · 브랜치 · 메인 파일(app.py) 선택.
4. [Advanced settings] → Secrets 칸에 `OPENAI_API_KEY = "sk-..."` 입력(10장 secrets.toml 과 같은 형식).
5. [Deploy] → 1~3분 뒤 앱 주소가 열린다. 코드를 GitHub 에 다시 올리면 자동으로 갱신된다.

자주 나는 오류

- ModuleNotFoundError → requirements.txt 에 라이브러리 이름을 빠뜨린 것
- FileNotFoundError → 내 PC 경로(C:₩Users₩...)를 쓴 것. `Path(__file__).parent / 'data'` 처럼 상대 경로를 쓴다(예제 코드 방식)
- 한글 그래프 깨짐 → 서버(리눅스)에는 맑은 고딕이 없다. 그래프 제목을 영어로 쓰거나 st.bar_chart 같은 Streamlit 차트를 쓴다

■ 확인 문제 (정답은 부록 A)

1. 배포 서버에 어떤 라이브러리를 설치할지 알려 주는 파일은?
2. 배포한 앱에서 OpenAI 키는 어디에 입력하는가?

APPENDIX A

부록 A. 확인 문제 정답

각 장 끝 확인 문제의 정답과 짧은 풀이

1장 Streamlit 시작하기

Q Streamlit 앱을 실행하는 명령어는 무엇인가?

A `streamlit run 파일이름.py`

Q 입력창의 값을 바꾸면 Streamlit 은 코드의 어느 부분을 다시 실행하는가?

A 파일 전체를 맨 위부터 다시 실행합니다.

2장 글자와 숫자 보여 주기

Q 표·리스트·숫자·문자열을 모두 출력할 수 있는 함수 하나는?

A `st.write()`

Q `st.metric('기온', '21 °C', '-1.2 °C')` 의 변화량은 어떤 색으로 표시되는가?

A 음수(–)이므로 빨간색 ↓ 로 표시됩니다.

3장 입력 위젯

Q `st.slider('나이 범위', 0, 80, (20, 40))` 의 반환값은 어떤 모양인가?

A (20, 40) 처럼 두 숫자로 된 튜플(범위)입니다.

Q 앞의 st03 예제에서 체크박스를 체크하지 않고 [제출] 을 누르면 어떤 상자가 나오는가?

A 노란 경고 상자 '동의에 체크해 주세요.' 가 나옵니다.

4장 화면 배치하기

Q 화면을 3:1 비율의 두 칸으로 나누는 코드는?

A `left, right = st.columns([3, 1])`

Q 로그인 입력처럼 '다 입력한 뒤 한 번에' 처리하려면 무엇으로 묶는가?

A `st.form` (안에 `st.form_submit_button` 사용)

5장 표와 차트

Q 사용자가 표의 값을 직접 고치게 하려면 어떤 함수를 쓰는가?

A `st.data_editor()`

Q matplotlib 으로 만든 fig 를 앱에 넣는 함수는?

A `st.pyplot(fig)`

6장 상태 기억하기 : session_state

Q 일반 변수 `count = 0` 으로 만든 카운터가 1 이상 올라가지 않는 이유는?

A 버튼을 누를 때마다 파일 전체가 다시 실행되면서 `count = 0` 도 다시 실행되기 때문입니다.

Q 위젯에 key='nick' 을 주면 그 값은 어디에서 꺼낼 수 있는가?

A `st.session_state.nick` (또는 `st.session_state['nick']`)

7장 캐싱으로 빠르게

Q 학습된 Keras 모델을 한 번만 불러오려면 어떤 데코레이터를 쓰는가?

A `@st.cache_resource`

Q `@st.cache_data` 함수의 저장된 결과를 지우는 방법은?

A `함수이름.clear()` (전체는 `st.cache_data.clear()`)

8장 파일 올리고 내려받기

Q 파일을 아직 올리지 않았을 때 아래 분석 코드가 실행되지 않게 하려면?

A `if file is None: st.stop()`

Q 다운로드 버튼에 넘기는 데이터는 어떤 형태여야 하는가?

A 문자열이나 bytes(예: `csv문자열.encode('utf-8-sig')`) 형태입니다.

9장 여러 페이지 앱

Q pages 폴더의 파일 '3_결과보기.py' 는 메뉴에 어떤 이름으로, 몇 번째로 나오는가?

A '결과보기' 라는 이름으로, 1·2 다음인 세 번째에 나옵니다.

10장 채팅 앱과 ChatGPT

Q 챗봇이 앞의 대화를 기억하게 하려면 OpenAI 에 무엇을 보내야 하는가?

A 지금까지의 대화 기록 전체(messages 리스트)를 보냅니다.

Q API 키를 코드에 직접 쓰지 않고 넣는 Streamlit 의 방법은?

A `.streamlit/secrets.toml` 에 저장하고 `st.secrets['OPENAI_API_KEY']` 로 읽습니다(배포 시에는 Secrets 설정 칸).

11장 미니 프로젝트

Q 대시보드에서 필터를 바꿀 때마다 CSV 를 다시 읽지 않게 해 주는 코드는?

A `load()` 함수 위의 `@st.cache_data`

12장 인터넷에 배포하기

Q 배포 서버에 어떤 라이브러리를 설치할지 알려 주는 파일은?

A `requirements.txt`

Q 배포한 앱에서 OpenAI 키는 어디에 입력하는가?

A Streamlit Community Cloud 앱 설정의 Secrets 칸 (Advanced settings)

APPENDIX B

부록 B. Streamlit 함수 한 장 요약

이 교안에서 쓴 함수 모음 — 책상 옆에 두고 보세요

분류	함수	용도
실행	`streamlit run app.py`	앱 실행 (종료: Ctrl+C)
출력	`st.title / header / subheader`	제목
	`st.write / markdown / text / caption`	글
	`st.code / latex / divider`	코드 · 수식 · 구분선
	`st.metric(라벨, 값, 변화)`	숫자 카드
	`st.success / info / warning / error`	알림 상자
입력	`st.button / text_input / number_input`	버튼 · 글자 · 숫자
	`st.slider / selectbox / radio / multiselect`	선택
	`st.checkbox / date_input / file_uploader`	체크 · 날짜 · 파일
배치	`st.set_page_config(layout='wide')`	페이지 설정
	`st.sidebar / columns / tabs / expander / form`	화면 나누기
데이터	`st.dataframe / table / data_editor`	표
	`st.bar_chart / line_chart / pyplot`	차트
상태	`st.session_state`	재실행되어도 값 유지
	`on_click=함수, key='이름'`	콜백 · 위젯 이름
속도	`@st.cache_data / @st.cache_resource`	결과 · 객체 저장
흐름	`st.stop() / st.rerun()`	여기서 멈춤 · 다시 실행
파일	`st.download_button / st.image`	내려받기 · 이미지
채팅	`st.chat_message / chat_input`	채팅 화면
비밀	`st.secrets['KEY']`	.streamlit/secrets.toml 값